BearLibTerminal / API / Configuration

Configuration is handled by set and get functions and by configuration file.

set

This is probably the most complex function in BearLibTerminal API. Configuring library options and mechanics, managing fonts, tilesets and even configuration file is performed with it:

```
terminal_set("window: size=80x25; font: ./UbuntuMono-R.ttf, size=12");
```

The function returns a boolean value indicating success or failure.

It is possible to set several options at once. However, this function tries to behave in transaction-like way: in case of error, it does not change anything and return false.

Configuration string format

A string passed to the function specifies a set of key-value parameters separated by semicolons:

```
window.title='foo'; window.size=80x25;
```

Related parameters can be grouped together:

```
window: title='foo', size=80x25;
```

Extra spaces, commas and semicolons are ignored. For now, all newline characters are also ignored, even inside quoted values, but this may change.

Quoting of string values is mostly optional and required only if value contains some delimiter characters like semicolons or you want to preserve leading/trailing whitespaces. Quoting may be done with both single and double quotes. There is no escape sequences besides Pascal-like double-quote escaping:

```
window.title='I''m feeling lucky!';
```

Library configuration

There is a number of options directly affecting library behavior and/or window appearance:

Group	Option	Default	Description
terminal	encoding	utf8	The encoding/codepage used for unibyte strings. It may be set to some ANSI codepage, e. g. Windows-1251.
window	size	80×25	The size of a window in cells.
	cellsize	auto	The size of a cell in pixels or “auto” if the size should be selected based on the font.
	title	'BearLibTerminal'	The title of the library window.
	icon	-	The name of an *.ico file to use for the library window. For Windows, default icon is built into the library.
	resizeable	false	The flag controlling window resizeability. If set, user can change window size and the application will receive a TK_RESIZED event.

	fullscreen	false	The virtual flag allowing to manually set fullscreen mode.
input	filter	'keyboard'	The list of events the application is interested in. All other will be silently processed by the library (more...).
	precise-mouse	false	When false, TK_MOUSE_MOVE event will be generated only when cursor is moved from one cell to another. When true, any mouse movement will generate the event.
	mouse-cursor	true	The flag controlling mouse cursor visibility.
	cursor-symbol	0x5F	The character used as a cursor symbol by read_str function.
	cursor-blink-rate	500	The rate cursor symbol blinks at in read_str function, in milliseconds.
	alt-functions	true	If set, the library will intercept some Alt-key combinations, e. g. Alt+Enter (toggling fullscreen mode) or Alt+Plus/Minus/Zero (zooming).
output	postformatting	true	Enables processing of special tags by print function.
	vsync	true	Switches on an off vertical synchronization used by OpenGL.
	tab-width	4	The width of a tab space in a text output by print function.
log	file	'bearlibterminal.log'	The name of a log file. Default is “bearlibterminal.log”.
	level	error	The logging level, one of “none”, “fatal”, “error”, “warning”, “info”, “debug”, “trace”.
	mode	truncate	The log writing mode: “truncate” will restart log each time, “append” will continue to write to a single file.

Font and tileset management

Beside the few fixed groups of library options, there may be any number of groups representing a font, tile or tileset in the following format:

```
[name ](font|offset): resource, param=value, ...;
```

Some examples would be:

```
font: UbuntuMono-R.ttf, size=12;
runic font: runic.png, size=8x16, codepage=437;
0x5E: curcumflex.png;
0xE000: tileset.png, size=16x16, spacing=2x1;
```

name

An optional name of some alternative font face, e. g. 'italic', 'runic', etc. Named fonts allow to use different styles of the same printable characters which will be available through `[font=name]` tag in the print function:

```
terminal_print(2, 1, "Its eyes are [font=italic]glowing[/font].");
```

offset

A number that specifies a code point of a single tile or a starting code point of a tileset. Individual tiles within a tileset are placed consecutively, so if tileset offset is 0xE000, then the first tile is 0xE000, the second one is 0xE001 and so on. Note that font characters and tiles share the same Unicode code space. Output functions (put, print) do not differentiate between 'characters', 'tiles' or 'sprites', they just use Unicode code points and the library renders whatever it is currently mapped to those code points:

```
terminal_set("0x40: at.png, align=center"); // Replace '@' (ASCII code 0x40) with a better version.
```

If you do not want for graphical tiles to overlap with text characters, it is recommended to use the specially reserved Unicode Private Use Area (0xE000-0xEFFF) code range for tileset offsets.

resource

Resource describes the font, tile or tileset data. It may be a filename or a memory buffer description in `address:size` format:

```
terminal_setf("font: %p:%d, size=8x16, codepage=437", buffer, size);
```

It is also possible to load a tileset from an array of BGRA pixels. In this case you must specify the size of the raw image with 'raw-size' parameter:

```
terminal_setf("0xE000: %p:%d, raw-size=%dx%d", pixels, W*H*4, W, H);
```

params

The number of parameters and their meaning is different depending on resource format: bitmap image or truetype font.

Type	Option	Default	Description
Bitmap	size		The size of a single tile in a tileset image. If not specified, the whole image is treated as one sprite.
	resize		The size to which tiles (or sprite) should be resized.
	resize-filter	bilinear	The resizing filter: “nearest”, “bilinear” or “bicubic”.
	resize-mode	stretch	The resizing aspect mode: “stretch”, “fit” or “crop”.
	raw-size		The dimensions of a raw memory image and it must be specified when loading from a pixel array.
	codepage	ascii	The tileset codepage, most useful for fonts.
	align	center	The tile aligment for the tileset: “center”, “top-left”, “bottom-left”, “top-right” or “bottom-right”.
	spacing	1x1	The tile alignment area, in cells.
	transparent	auto	The color of a background for image formats not supporting transparency.
TrueType	size		The size of a symbol or tile. This option is mandatory.
	size-reference	'@'	The symbol used for size probing.
	mode	normal	Rasterization mode: “monochrome”, “normal” or “lcd”. Note that “lcd” forces an opaque black background.
	codepage		The reverse codepage for loading a select set of symbols.
	align	center	The tile alignment, same as for bitmap tilesets.
	spacing	1x1	The tile alignment area, same as for bitmap tilesets.
	use-box-drawing	false	Use font's Box Drawing characters, generate them otherwise.
	use-block-elements	false	Use font's Block Elements characters, generate them otherwise.
	hinting	normal	Font hinting: “normal” (font's native hinter), “autohint” (FreeType's autohinter) or “none” (no hinting).

Font and tileset options are set or replaced altogether. All relevant options for a tileset must be specified in a single **set** call.

Bitmap tileset description examples:

```
font: somefont.png, size=8x16, codepage=1251;
0xE000: walls.bmp, size=32x32, spacing=4x2;
0xE100: background.jpg, resize=640x400, resize-filter=bicubic, align=top-left;
```

TrueType tileset description examples:

```
font: somefont.ttf, size=12;
0xE000: otherfont.ttf, size=8x16, codepage=1251;
0xE100: fontawesome.ttf, size=32x32, spacing=4x2, codepage=fontawesome-cp.txt;
```

To remove the tileset, set the main value of its option group to 'none' (no other properties required in this case):

```
0xE000: none
```

Modifying configuration file

It is possible to add, modify or remove an option from the library's configuration file by providing its name in the `ini.section.property` format:

```
ini.game.tile-size = 16
```

In the example above, the library will modify the 'tile-size' property in 'game' section.

If there is no such property or section in the configuration file, it will be added. To remove a property from the file, specify an empty value. This means it is impossible to programmatically set property to an empty string, so use some meaningful default value instead.

get

This function retrieves the value of an option (a library option or a property from the configuration file). If no such option is available, the supplied default value is returned:

```
const char* tile_size = terminal_get("ini.game.tile-size", "24");
```

Right now only library and configuration options may be queried, retrieving font/tileset properties is not supported yet.

In C/C++ interface, memory of the returned string is managed by the library and remains valid until the next **get** call with the same option/property name (but immediately copying the value is encouraged). The return value will always be a valid C string, even if there is no such option and no default value was provided (an empty string is returned in this case).

Configuration file

Since 0.12, a special configuration file may be used to provide initial library configuration and a storage for arbitrary application-specific options.

This file must be in the common INI file format with few minor peculiarities:

- Both `#` and `;` may be used for comments. Lines starting from whitespace are also ignored.

- Section and property names are not case sensitive (but library tries to preserve the casing when modifying).
- Duplicate (reopened) sections are allowed.
- The library does not understand slash-escaping, “\n” is a string of two regular characters.
- Whitespace around property name or value is ignored.
- Property line may contain a group of properties (see configuration string format).

An example of configuration file would be:

```
# Line comment
; Another line comment
  This will also be treated as comment

[BearLibTerminal]
log: file=bt.log, level=trace
font: UbuntuMono-R.ttf, size=12
window.title='Configuration file example'

[Game]
tile-size = 16 ; Everything after semicolon is ignored
```

Configuration file name is not fixed, the library will search for a suitable file. Unless there is an file named after the application, first available INI file will be used. Files from current working directory has higher priority, but the directory with executable is also searched. Therefore it is possible to reuse an existing INI file.

When terminal instance being initialized (via the call to terminal_open), the library will search for configuration file. If found, it will use “BearLibTerminal” section to perform initial configuration, then proceed as usual. During initial configuration logging options are applied first. Everything else is grouped and applied on per-group basis in separate set calls, so that if there is some problem with particular option, it will not cause library to reject entire configuration.